GUIDE TECHNIQUE & IMPLÉMENTATION // RÉF. OKOS-TTD-01

TIME-TRAVEL DEBUGGING : ANNULER ET RÉTABLIR LES ÉTATS DU CUBE 3D (256 BITS)

Spécification algorithmique de la machine à remonter le temps logique, journalisation transactionnelle SQLite et réversibilité temporelle du micro-noyau **Oracle Kernel OS (< 1 Mo)**.

ARCHITECTE : DAVID TREMBLAY-DUVAL MÉCANISME : HISTORIQUE TRANSACTIONNEL DATE : OCTOBRE 2026

1. Le Concept de Time-Travel Debugging (Voyage Temporel Logique)

SECTION 01 // PARADIGME

En physique, le voyage temporel vers le passé demeure une conjecture spéculative. En architecture système et en génie logiciel, en revanche, la **réversibilité temporelle** (*Time-Travel Debugging*) est une réalité mathématique déterministe. Dans l'environnement d'**Oracle Kernel OS**, le Cube VFS 3D de 256 bits et son moteur de persistance SQLite forment un journal d'états inaltérable. En enregistrant chaque mutation de voxel comme un vecteur horodaté doté d'un index séquentiel $\mathbf{T_n}$, le système permet non seulement de connaître l'état présent de la matrice, mais aussi de « rembobiner » ou de « avancer » instantanément à n'importe quel point de l'historique d'exécution.

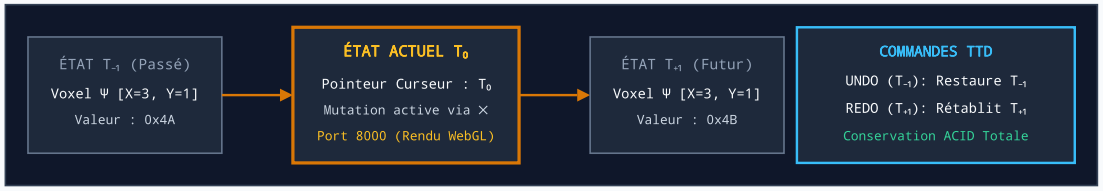


Figure 1.1 — Déplacement du curseur temporel au sein de la table des historiques d'états SQLite.

2. Modélisation Mathématique de l'Historique d'État

SECTION 02 // FORMALISME

Soit $\mathbf{S_t}$ l'état global de la matrice du Cube à l'instant logique $\mathbf{t} \in \mathbb{N}$. Chaque opération d'écriture ou de mutation quaternaire sur un voxel `Ψ(X, Y, Z)` génère une transition d'état formelle $\boldsymbol{\delta} : \mathbf{S_t} \rightarrow \mathbf{S_{t+1}}$. Au lieu d'écraser physiquement les anciennes données dans la base de données SQLite, le système conserve l'intégralité des deltas transactionnels dans une table d'historique circulaire $\mathscr{H} = \{ (t, X, Y, Z, V_{old}, V_{new}) \}$. Les fonctions d'annulation (`UNDO`) et de rétablissement (`REDO`) s'expriment par conséquent comme des opérateurs de translation inverse sur le curseur temporel $\boldsymbol{\tau}$.

3. Implémentation SQLite & Schéma de la Table d'Historique SECTION 03 // DONNÉES

Pour rendre cette machine à remonter le temps opérationnelle dans un binaire de moins de 1 Mo sans dépendance externe, le schéma SQLite intègre une table dédiée aux journaux d'annulation. Cette table mémorise l'empreinte des 256 bits de chaque voxel avant et après modification, garantissant un retour en arrière instantané sur simple requête SQL :

Champ / Colonne	Type SQLite	Rôle Fonctionnel dans le Mécanisme de Voyage Temporel
`id`	`INTEGER PRIMARY KEY`	Identifiant séquentiel unique de la transaction (index temporel absolu **t**).
`voxel_x, y, z`	`INTEGER`	Coordonnées spatiales tridimensionnelles du voxel ciblé dans le Cube 3D.
`old_payload`	`BLOB (32 octets)`	Valeur précédente du registre de 256 bits (nécessaire pour exécuter `UNDO`).
`new_payload`	`BLOB (32 octets)`	Nouvelle valeur du registre de 256 bits après mutation (nécessaire pour `REDO`).
`timestamp`	`TEXT / REAL`	Horodatage précis de la transition d'état au sein du micro-noyau.
Curseur τ	**Pointeur Dynamique**	Indique la position active actuelle dans l'historique lors des phases de navigation.

4. Algorithmes d'Annulation (UNDO) et de Rétablissement (REDO) SECTION 04 // ALGORITHMIQUE

La logique de navigation temporelle repose sur le déplacement d'un pointeur de curseur temporel **τ**. Lorsqu'un utilisateur ou un script demande une annulation via le shell du port `9000`, le noyau extrait le delta précédent de la table d'historique et réinjecte l'ancien contenu du registre de 256 bits dans la matrice active, tout en mettant à jour le rendu graphique 3D sur le port `8000`.

SÉQUENCE D'EXÉCUTION DE L'OPÉRATEUR UNDO (REMBOBINAGE D'UN VOXEL)

1. **Vérification du Curseur :** Si `τ > 0`, le système cible la transaction T_τ dans la table d'historique.
2. **Restauration de la Mémoire Vive :** Le registre du voxel `(X, Y, Z)` est écrasé par `old_payload`.
3. **Mise à Jour SQLite & Décrémentation :** L'état est persisté et le pointeur temporel devient `τ ← τ − 1`.
4. **Notification WebGL :** Le port `8000` diffuse immédiatement la mise à jour visuelle du cube.

5. Intégration Code Source (Python Standard) & Commandes Shell

SECTION 05 // CODE

Conformément au cahier des charges d'**Oracle Kernel OS**, cette fonctionnalité est implémentée en pur Python sans aucune bibliothèque externe, en s'appuyant exclusivement sur le module natif `sqlite3` pour garantir la portabilité du binaire `.exe` :

Fonction UNDO

RETOUR PASSÉ

`def time_undo(cursor):`
Récupère la dernière transaction enregistrée, réapplique `old_payload` au voxel spatial, décrémente l'index temporel et rafraîchit la matrice 3D instantanément.

Fonction REDO

AVANCÉE FUTUR

`def time_redo(cursor):`
Parcourt l'historique vers l'avant, réapplique `new_payload` au voxel cible et repositionne le curseur temporel `τ` sur l'état avamcé souhaité.

Commandes Shell

PORT 9000

Via le terminal système, les commandes `/time undo` et `/time redo [steps]` permettent de piloter la machine temporelle interactivement et d'auditer l'évolution de l'OS.

Auditabilité Totale

SANS LATENCE

L'indexation par SQLite garantit que même après des millions de mutations, le parcours temporel reste instantané et totalement étanche aux corruptions de mémoire vive.

6. Conclusion : La Maîtrise Totale du Temps Système

SECTION 06 // SYNTHÈSE

En dotant le Cube VFS 3D de 256 bits d'un journal d'historique transactionnel réversible, **Oracle Kernel OS** transcende la simple fonction de stockage pour devenir un système à l'épreuve de l'erreur humaine. Qu'il s'agisse de déboguer une séquence d'instructions en environnement tactique isolé ou d'annuler une modification critique sur le terrain, la machine temporelle logique offre aux opérateurs une maîtrise absolue sur le flux des données.

PROPRIÉTÉ INTELLECTUELLE & LICENCE D'EXPLOITATION EXCLUSIVE
Copyright © 2026 **David Tremblay-Duval**. Tous droits réservés (*All Rights Reserved*). L'architecture de rétro-ingénierie temporelle (*Time-Travel Debugging*) appliquée au Cube VFS 3D, le micro-noyau **Oracle Kernel OS** et les spécifications associées sont la propriété intellectuelle exclusive de l'auteur. Toute reproduction ou intégration non autorisée est strictement interdite. Document officiel exporté pour le dépôt GitHub et les dossiers institutionnels (IDEeS / AID).